

PYTHON FOR DATA SCIENCE • MODULE 1 • TOPIC 1.3

Functions & Scope

Day 3 — writing code once and reusing it everywhere

90 min

Concepts + live code

30 min

Practice notebook

4

Ways to pass arguments

Prepared & presented by **Srinivasa Sai Chava**



Boston University

Before we start — five questions from Day 2

1 **range(1, 5) gives which numbers?**

→ 1, 2, 3, 4 — the stop value is excluded.

2 **Does continue stop the loop?**

→ No. It skips one item. Only break stops it.

3 **Why do three separate ifs go wrong?**

→ All are tested; each match overwrites the last. Use elif.

4 **What makes a while loop run forever?**

→ No update inside the body. Press stop in Colab.

5 **for n in nums: n = n*2 — what changes?**

→ Nothing. n is a copy; append to a new list.

A function body is just another indented block — yesterday's indentation work carries straight over.

How today's two hours are structured

1

Defining & calling

def, parameters, return vs print

20 min

2

Arguments

Positional, keyword, defaults, *args, **kwargs

30 min

3

Scope

Local vs global, the LEGB rule

25 min

4

Writing them well

Docstrings, type hints, lambda

10 min

5

Mini build

Refactoring Day 1's result calculator

5 min

6

Practice notebook

You code, I walk around

30 min

Section 2 is the long one — five different ways to hand data to a function, and one famous trap.

By the end of today you will be able to

1

Define

Write a function with `def`, call it, and return a value from it

2

Pass

Use positional, keyword and default arguments — and `*args / **kwargs`

3

Reason

Predict which variables a function can see, and which it cannot

4

Refactor

Take a repetitive script and reshape it into clean, reusable functions

The habit to build today

Write the function signature and the docstring FIRST, before any logic. Deciding what goes in and what comes out — before you write the body — is most of the design work. It also stops you writing a function that quietly does three unrelated things.

1

Defining & Calling

Packaging code under a name · 20 minutes

- Why functions
- def anatomy
- return vs print
- Multiple returns
- Early return

Why functions? Because copy-paste does not scale

Without a function

```
# Ravi
total = 88 + 71 + 64
pct   = total / 300 * 100
print(f"Ravi: {pct:.1f}%")

# Sara
total = 91 + 84 + 79
pct   = total / 300 * 100
print(f"Sara: {pct:.1f}%")

# ...and again for every student
```

With a function

```
def percentage(a, b, c):
    total = a + b + c
    return total / 300 * 100

print(f"Ravi: {percentage(88,71,64):.1f}%")
print(f"Sara: {percentage(91,84,79):.1f}%")

# The formula is written ONCE.
# Fix it once, and every caller is fixed.
```

Don't Repeat Yourself

One definition, many uses

One place to fix

A bug lives in one spot, not twenty

Readable

percentage(...) says what it does

Testable

You can check one piece in isolation

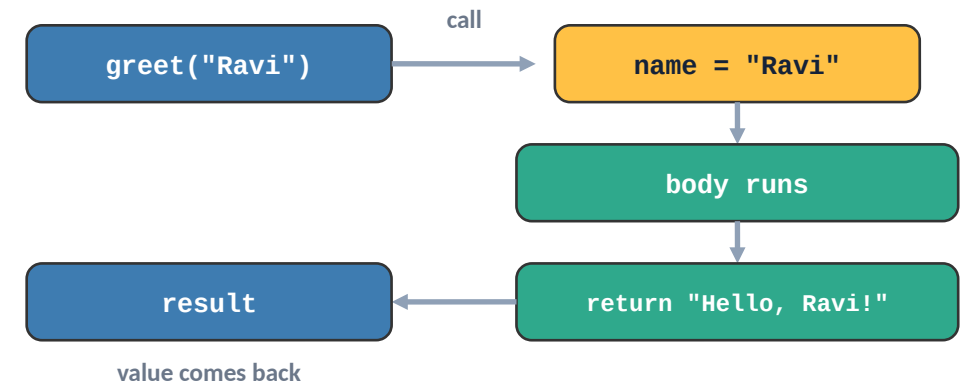
Anatomy of a function

```
def greet(name):
    message = f"Hello, {name}!"
    return message

result = greet("Ravi")
```

def	the keyword that starts a definition
greet	the function's name (snake_case)
(name)	the parameter list
:	opens the block — body is indented
return	sends a value back to the caller

What happens when you call it



Defining is not running

```
def greet(name):          # defining: nothing happens yet
    print("Hello")

greet("Ravi")             # calling: NOW the body runs
```

A very common first-week confusion: students write a perfect function, run the cell, see no output, and assume it is broken. The definition only teaches Python the recipe. Nothing cooks until you call it — and the parentheses are what make it a call.

return vs print — the distinction that matters most today

print — shows text on screen

```
def add(a, b):  
    print(a + b)          # shows it  
  
result = add(2, 3)        # displays 5  
print(result)             # None !  
  
print(result * 2)  
    -> TypeError: unsupported operand  
  
# The value was never handed back.
```

return — hands the value back

```
def add(a, b):  
    return a + b          # gives it back  
  
result = add(2, 3)        # nothing displayed  
print(result)             # 5  
  
print(result * 2)         # 10 - it is a real number  
total = add(1,2) + add(3,4) # composable  
  
# The value can now be USED.
```

The rule: `print` is for a human to read. `return` is for the rest of your program to use. Almost every function you write should `return`.

A function with no return statement returns `None` automatically. That `None` is where the mysterious “`TypeError: unsupported operand type(s) for *: 'NoneType'`” comes from — you tried to do arithmetic on nothing.

Three things return does that surprise people

1 return exits immediately

```
def check(n):  
    return "positive"  
    print("never runs")    # unreachable  
  
# Nothing after return is executed.
```

2 You can return several values

```
def stats(nums):  
    return min(nums), max(nums),  
    sum(nums)  
  
lo, hi, tot = stats([3, 9, 4])  
# Python packs them into a tuple,  
# then unpacks - Day 1's swap trick.
```

3 Early return simplifies logic

```
def grade(m):  
    if m < 0 or m > 100:  
        return "invalid"    # leave  
    now  
    if m >= 75:  
        return "A"  
    return "F"
```

Early return replaces deep nesting

Handle the invalid and edge cases first, return out of them, and the rest of the function can assume clean input. This keeps the main logic at one indent level instead of three — much easier to read and far easier to debug.

2

Arguments

Five ways to hand data to a function · 30 minutes

- Positional
- Keyword
- Defaults
- The mutable trap
- `*args / **kwargs`

Positional and keyword arguments

Parameters are the names in the definition. **Arguments** are the actual values you pass in when calling.

```
def introduce(name, age, city):      # name, age, city are PARAMETERS
introduce("Ravi", 23, "Pune")      # those three values are ARGUMENTS
```

Positional — order is everything

```
introduce("Ravi", 23, "Pune")      # correct

introduce(23, "Ravi", "Pune")      # NO ERROR
    -> name=23, age="Ravi"

# Python cannot tell you got it wrong.
```

Keyword — order stops mattering

```
introduce(name="Ravi", age=23, city="Pune")
introduce(city="Pune", name="Ravi", age=23)

# Both identical. Self-documenting.

introduce("Ravi", city="Pune", age=23) # mix ok
```

Rule when mixing: every positional argument must come before every keyword argument. Reverse that and you get a `SyntaxError`.

Default arguments — making a parameter optional

Give a parameter a fallback value

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

greet("Ravi")           # Hello, Ravi!
greet("Ravi", "Welcome") # Welcome, Ravi!
greet("Ravi", greeting="Hi") # Hi, Ravi!
```

Why they matter

- Simple calls stay simple; advanced options stay available
- You can add a new parameter without breaking existing calls
- This is why `train_test_split(X, y)` works with no other arguments — everything else has a default



Defaults must come AFTER non-default parameters

```
def greet(greeting="Hello", name):
    -> SyntaxError: non-default argument
        follows default argument
```

Python fills positional arguments left to right, so it cannot work out what a bare value means once an optional parameter sits ahead of a required one. Required parameters first, optional ones after — always.

The famous mutable default trap

! This looks completely reasonable

```
def add_item(item, basket=[]):  
    basket.append(item)  
    return basket  
  
add_item("apple")      # ['apple']  
add_item("bread")     # ['apple', 'bread'] ?!  
add_item("milk")       # ['apple', 'bread', 'milk']  
  
# Each call was supposed to start empty.
```

The fix: default to None

```
def add_item(item, basket=None):  
    if basket is None:  
        basket = []      # fresh list per call  
    basket.append(item)  
    return basket  
  
add_item("apple")      # ['apple']  
add_item("bread")     # ['bread'] correct
```

Why it happens — and it is Day 1's lesson again

The default value is created ONCE, when the function is defined — not on each call. So every call that omits the argument shares the exact same list object. This is aliasing: two names pointing at one object, exactly like `c = a` on Day 1. Never use a list, dict or set as a default value. Use `None` and build it inside.

*args and **kwargs — accepting any number of arguments

***args** → collects extras into a TUPLE

```
def total(*args):
    print(args, type(args))
    return sum(args)

total(1, 2)          # (1, 2)      -> 3
total(1, 2, 3, 4)    # (1,2,3,4)  -> 10
total()              # ()         -> 0

# The * is the syntax. 'args' is just a
# conventional name - *nums works too.
```

****kwargs** → collects extras into a DICT

```
def profile(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

profile(name="Ravi", age=23)
name: Ravi
age: 23

# kwargs is a normal dict - Topic 1.5
```

The order is fixed: positional → *args → keyword → **kwargs

```
def f(a, b, *args, sep=", ", **kwargs):
    ...
```

You will meet these constantly in library code — it is how a function like `plt.plot()` accepts almost anything. Writing them yourself is rarer, but reading them is a daily skill.

3

Scope

Which variables can a function see? · 25 minutes

- Local vs global
- The LEGB rule
- The global keyword
- Mutable arguments

Local and global scope

Variables made inside a function stay inside

```
course = "Data Science"      # GLOBAL

def show():
    room = "B-204"           # LOCAL
    print(course)             # can READ global
    print(room)

show()                        # works
print(room)
-> NameError: name 'room' is not defined
```

Two nested boxes

GLOBAL (the whole notebook)

course = "Data Science"

LOCAL (inside show)

room = "B-204"

↑ can see out to global

↓ global cannot see in

Why this is a feature, not a restriction

Because locals are sealed off, you can use `total` as a variable name inside fifty different functions and they will never collide. Each call gets its own private workspace that is created on entry and thrown away on return. Without scope, every function in every library you import could clobber your variables.

The LEGB rule — how Python resolves a name



Local

Inside the current function



Enclosing

Inside any outer function wrapping it



Global

Top level of the file or notebook



Built-in

Python's own names: len, print, sum, range

Shadowing in action

```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)      # local
    inner()
    print(x)          # enclosing

outer()
print(x)              # global
```

Python searches in this order and stops at the FIRST match.

This is why naming a variable `list` or `sum` breaks things: your Global name is found before Python's Built-in, so the original function is no longer reachable. Day 1's shadowing warning is just the B in LEGB losing to the G.

Changing outer variables — and when it happens by accident

Assigning creates a LOCAL, always

```
count = 0

def increment():
    count = count + 1
    -> UnboundLocalError

# Assigning anywhere in the body makes
# count local for the WHOLE function,
# so the right-hand side reads a local
# that does not exist yet.
```

Better: take it in, hand it back

```
def increment(count):
    return count + 1

count = increment(count)    # explicit

# The 'global' keyword also works:
#     def increment():
#         global count
#         count += 1
# but it hides where the change came from.
```

⚠ But mutable objects CAN be changed from inside — no global needed

```
def add_one(items):
    items.append(1)    # MUTATES the caller's list
```

Rebinding a name (=) is local. Mutating an object (.append, [i] = x) reaches through to the original, because both names point at the same object. Day 1's aliasing, one more time — it is the single idea behind this slide, the mutable-default trap, and the loop-variable surprise from Day 2.

4

Writing Them Well

Docstrings, type hints, lambda · 10 minutes

- Docstrings
- Type hints
- One job per function
- lambda

Docstrings and type hints

The professional shape of a function

```
def percentage(marks: list, out_of: int = 100) -> float:
    """Return the percentage score for a list of marks.

    Args:
        marks: list of individual subject marks
        out_of: maximum marks per subject (default 100)

    Returns:
        The percentage as a float, e.g. 74.33
    """
    return sum(marks) / (len(marks) * out_of) * 100
```

Why bother

- `help(percentage)` prints your docstring
- In Colab, Shift+Tab shows it as you type the call
- Type hints are documentation, not enforcement — Python does not check them
- Editors use hints to catch mistakes before you run anything

One function, one job. If your docstring needs the word “and” twice to describe what the function does, it is really two functions. A function that loads a file, cleans it, trains a model and prints a chart is impossible to test and impossible to reuse.

lambda — a one-line function with no name

Same thing, two ways

```
def square(n):  
    return n * n  
  
square = lambda n: n * n  
  
# lambda <params>: <single expression>  
# The result is returned automatically.
```

Where you will actually meet it

```
students = [("Ravi", 88), ("Sara", 91)]  
  
students.sort(key=lambda s: s[1])  
#                ^ sort by the score  
  
df["col"].apply(lambda x: x * 2) # pandas
```

Use it as an argument to another function — not as a replacement for def

```
square = lambda n: n * n      # pointless - just use def  
sort(key=lambda s: s[1])     # exactly right
```

A lambda holds one expression and cannot contain statements — no loops, no if blocks, no multiple lines. If you find yourself fighting that limit, you wanted a real function. Its purpose is to describe a small piece of behaviour at the moment you hand it to something else, and you will use it constantly with pandas in Topic 1.15.

Debug corner — the eight function bugs you will actually hit

You wrote	What happens	Fix
<code>print inside, no return</code>	caller gets None	<code>return the value</code>
<code>greet</code>	nothing runs	<code>greet()</code>
<code>def f(x=[]) ... x.append</code>	list grows across calls	<code>x=None, then x = []</code>
<code>def f(a=1, b)</code>	SyntaxError	defaults go last
<code>count = count + 1</code>	UnboundLocalError	pass it in and return it
<code>code after return</code>	never runs	move it before the return
<code>f(23, "Ravi")</code>	wrong order, no error	use keyword arguments
<code>items.append()</code> in a function	changes the caller's list	pass a copy: <code>items[:]</code>

All eight appear in today's practice notebook. Rows 1 and 5 account for most of the questions I get during practice.

Putting it together — refactoring Day 1's result calculator

```
def percentage(marks, out_of=100):  
    """Return the percentage for a list of marks."""  
    return sum(marks) / (len(marks) * out_of) * 100  
  
def grade(pct):  
    """Map a percentage to a letter grade."""  
    if not 0 <= pct <= 100:  
        return "invalid"        # early return  
    if pct >= 75: return "A"  
    elif pct >= 60: return "B"  
    elif pct >= 40: return "C"  
    return "F"  
  
def report(name, marks):  
    """Print one student's result line."""  
    pct = percentage(marks)  
    print(f"{name:8} {pct:6.2f}%   {grade(pct)}")  
  
students = {"Ravi": [88, 71, 64], "Sara": [91, 84, 79]}  
for name, marks in students.items():  
    report(name, marks)
```



Three small functions

Each does exactly one job



Docstrings

One line each is enough



Default argument

out_of=100 keeps calls short



Early return

Invalid input leaves immediately



Composition

report() calls the other two



Day 2's loop

for + .items() drives it all

Compare this with the single long block from Day 1. Same output — but now each piece can be tested, reused and fixed on its own.

Practice session — 30 minutes



Predict the output

Watch for None, scope and defaults

8 questions



Spot & fix the bug

print vs return, mutable default, UnboundLocal

4 questions



Write the code

Temperature, password check, stats, word count

5 tasks

Your notebook

[Day3_Practice_Questions.ipynb](#)

- Signature and docstring first
- Then the body
- Raise a hand — I am walking around
- Solutions shared after class

If your function “does nothing”, check two things: did you call it, and did you return rather than print?

TODAY IN TWELVE LINES

Recap

- 1 `def` defines; parentheses call. Both are needed.
- 2 `return` hands a value back; `print` only displays.
- 3 No `return` statement means the function returns `None`.
- 4 `return` exits the function immediately.
- 5 Return several values at once — they arrive as a tuple.
- 6 Keyword arguments make a call self-documenting.
- 7 Defaults go last, and must never be a list or dict.
- 8 `*args` collects a tuple; `**kwargs` collects a dict.
- 9 Locals are private; each call gets a fresh workspace.
- 10 LEGB: Local, Enclosing, Global, Built-in.
- 11 Assigning makes a local; mutating reaches the original.
- 12 One function, one job. Docstring first, body second.

Homework

Rewrite Day 2's guessing game using three functions · Write `is_valid_email(text)` returning `True/False` · Add docstrings to everything you wrote today

Next class — Topic 1.4

Lists, tuples and sets: creating, indexing, slicing, mutability, and the methods you will use on every dataset